DYLAN PIWKO

AEROSPACE ENGINEERING AT THE UNIVERSITY OF MICHIGAN

dpiwko@umich.edu
linkedin.com/in/dylan-piwko
dylanpiwko.github.io

SUMMARY

First-year aerospace engineering student at the University of Michigan and former NASA Ames concept design intern. I design and fly high-power rockets, including the avionics systems. At NASA I worked on drone route planning in urban airspace, and this fall I helped an Embry-Riddle team predict aircraft taxi times with machine learning. In January, I'll start work with Firefly Aerospace on the catch recovery system for the Eclipse launch vehicle as part of the Perot Jain TechLab.

PROJECTS

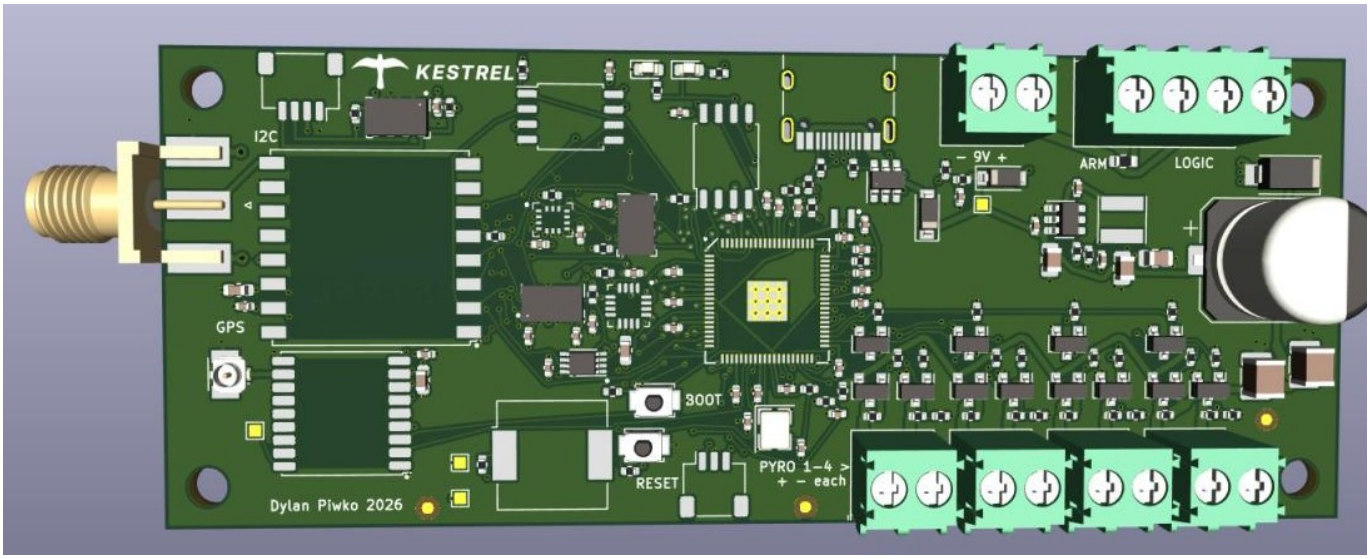
Project	Role	Skills and tools	Page
Kestrel Flight Computer	Designer, Personal Project	KiCad, 4-layer PCB design, LoRa telemetry, GPS, verification planning	2
Aeropulse, NASA Ames Research Center	Concept Design Intern	Systems design, UAS route planning, UTM and airspace integration, trade studies, technical writing	5
Terrier-Sandhawk Two-Stage Rocket	Builder and Flyer, Personal Project	RockSim, electronic staging, avionics integration, failure analysis	7
EUROCONTROL PRC Data Challenge	Team Member, Embry-Riddle Team	Python, LightGBM, CatBoost, SHAP, feature engineering	10



My Terrier-Sandhawk two-stage rocket at liftoff

KESTREL FLIGHT COMPUTER

Designer, Personal Project
October 2026 – Present



Kestrel in KiCad (100 × 40 mm, 4 layers)

What?

- Flight computer for my 4" Wildman Punisher 4, which I'm flying for my NAR Level 2 certification in January 2027
- It logs data and sends live telemetry, and will eventually handle dual-deployment recovery, including a later L2200G flight that simulates to about 14,800 ft and Mach 1.7

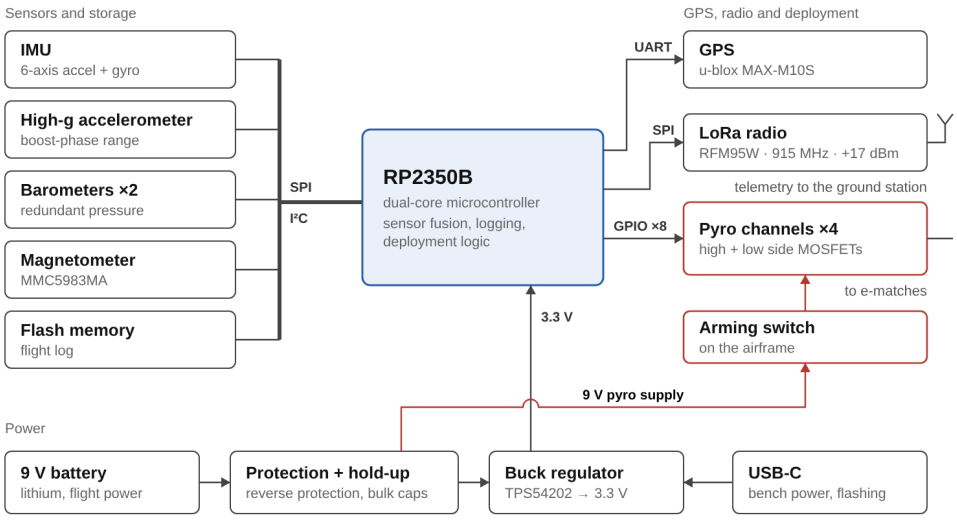
How?

- Designed the schematic and 4-layer board in **KiCad** (100 × 40 mm, 145 parts)
- Selected an RP2350B microcontroller, IMU, high-g accelerometer, two barometers, magnetometer, u-blox GPS, 915 MHz LoRa radio, and flash memory
- Switched each of the four pyro channels on both the high and low side, behind a physical arming switch
- Designed the power stage around a 9 V battery, with reverse-polarity protection and a TPS54202 buck converter for 3.3 V

Results

- Zero ERC and DRC errors, with every part checked pin by pin against its datasheet and Raspberry Pi's reference design
- Received a PCBWay Future Innovators sponsorship for the board order
- My commercial altimeters keep handling recovery until Kestrel passes ground and flight tests

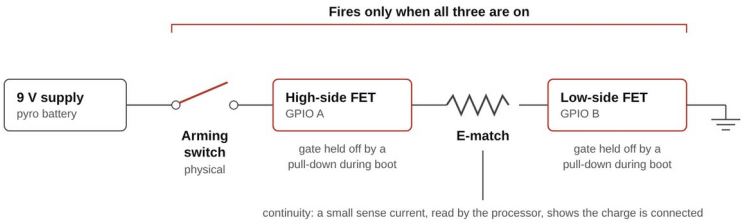
DESIGN AND VERIFICATION



Block diagram

I switched each e-match on both its high side and its low side, behind a physical arming switch on the airframe, so a charge can only fire when that switch is closed and the processor turns on both transistors.

Pull-down resistors keep every gate off while the processor boots, and a small sense current through each e-match shows continuity without firing it.

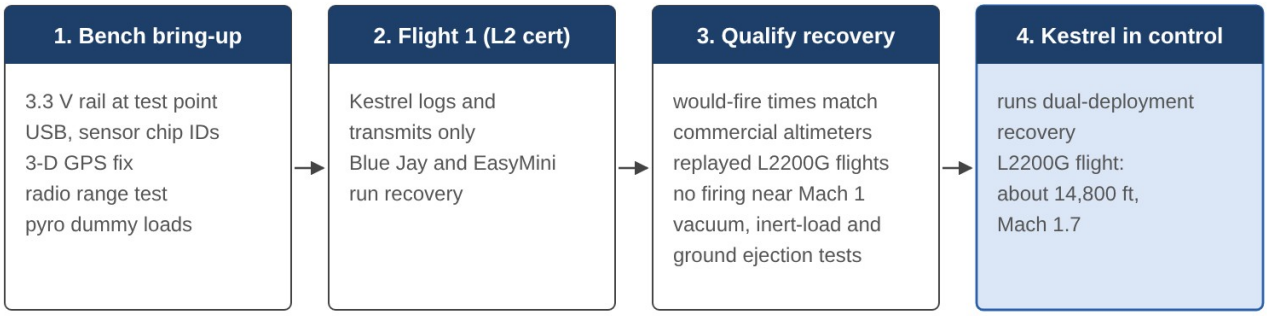


One of the four pyro channels

I can only afford one board revision, so before ordering I checked each subsystem against a datasheet or reference design and planned a test to prove it once the boards show up.

Subsystem	Checked against	Proven at bring-up by
Power	TI's TPS54202 3.3 V reference values; USB-only start-up margin	3.25–3.35 V at the test point
Processor	All 80 RP2350B pins against the pin table and SparkFun's production board	Enumerates over USB
Sensors and GPS	Each pinout and SPI mode against its datasheet; u-blox integration manual	Chip ID self-test; 3-D GPS fix
Radio	Pinout; link budget of about 29 dB margin at 10 km	Ground-station range test
Pyro	MOSFET ratings, gate drive, boot pull-downs	Dummy-load tests, armed and disarmed
Layout	Zero ERC/DRC errors; PCBWay engineering review	Visual inspection of both assembled boards

TEST PLAN



Test plan before Kestrel controls recovery

On the first flight Kestrel only logs and transmits. A Featherweight Blue Jay and an EasyMini handle recovery. Before I let it control a parachute, its would-fire times from flight 1 have to line up with when the commercial altimeters actually fired. It also has to fire correctly on replayed L2200G simulations without ever firing through the Mach transition. Then it goes through vacuum chamber, inert load, and ground ejection tests.

LONG-TERM GOALS

The end goal for this board is the L2200G flight, about 14,800 ft and Mach 1.7, with Kestrel firing all of the recovery charges.

After that, I want to lower the cost per board enough to build Kestrels for more than one rocket.

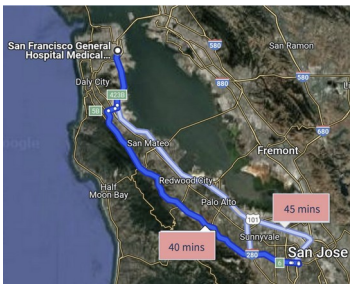
Eventually, I might try to turn Kestrel into a small business. I see a gap in the market for flight computers that are affordable but still capable enough for high-performance flights. Right now, a board with GPS tracking and live telemetry starts at $300, and getting four or more pyro channels too means buying the most expensive one or pairing two separate products:

Product	What it has	Price
Altus Metrum TeleMetrum	2 pyro channels, GPS, telemetry	$300
Altus Metrum TeleMega	6 pyro channels, GPS, telemetry	$400
Featherweight Blue Raven	4 pyro channels, IMU, no GPS	$194.49
Featherweight GPS system	GPS tracker and ground station, no pyro channels	$420

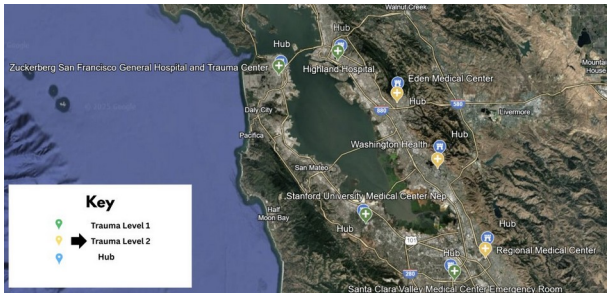
The Altus Metrum boards also need a separate receiver on the ground, and a Blue Raven plus a Featherweight GPS system comes to over $600. Prices are from each retailer's site in November 2026.

AEROPULSE - NASA AMES RESEARCH CENTER

Concept Design Intern, Aviation Systems Division
June – August 2025



Modeled route (map data © Google)



Proposed drone hubs (imagery © Google)



SkyLane hybrid VTOL, the drone we based Aeropulse on (photo © Sky-Drones)

What?

- Our team of six designed **Aeropulse**, a concept that moves organs, lab samples, and urgent medicine between hospitals with hybrid VTOL drones and stair-climbing robots, with no manual handoffs
- I was one of 36 students picked from about 800 applicants for the Aviation Systems Division's Concept Design Experience

How?

- I was responsible for drone routing and urban airspace integration
- Examined routing constraints, airspace rules (UTM, geofencing, detect-and-avoid), and trade-offs between the drones and the ground robots
- Conceptualized route planning for both vehicles, with 3-D occupancy-grid routing, GPS, and SLAM for the drones and A* and Dijkstra path planning for the robots
- Planned for operations below 400 ft AGL under **Part 107** with BVLOS waivers

Results

- In our model, a donor heart gets from San Francisco General to Santa Clara Valley Medical Center in about 37 min, compared with 70 to 90+ min today
- Presented the design to 50+ NASA staff
- Co-authored NASA/TM-20250009029 with the rest of the team

THE CONCEPT

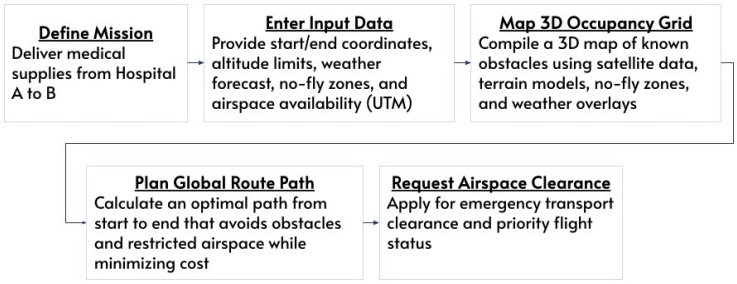


A robot carries the payload out of Hospital A, rides the drone, and takes it to the right room in Hospital B (from the memo)

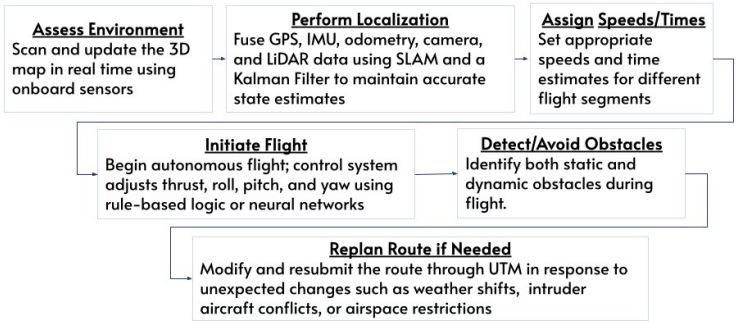
ROUTE PLANNING

Before takeoff, the drone plans its whole route from Hospital A to Hospital B. It takes in the start and end points, altitude limits, the weather forecast, no-fly zones, and open UTM airspace.

From those it builds a 3-D occupancy grid and finds the lowest-cost path around obstacles and restricted airspace. Then it asks UTM for emergency priority clearance.



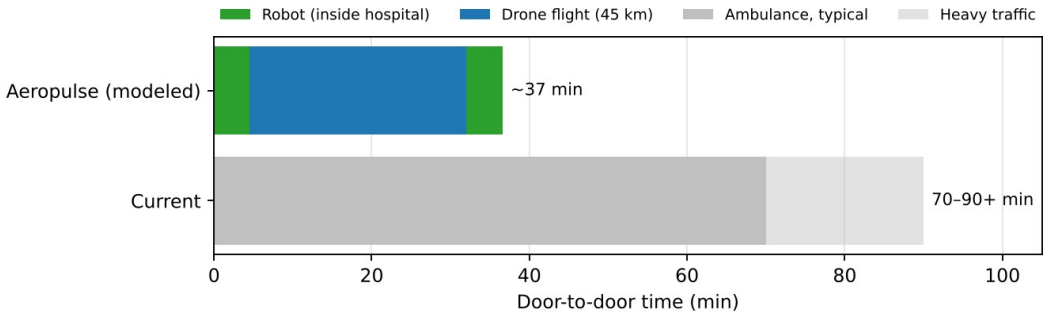
Pre-flight route planning (from the memo)



In-flight route planning (from the memo)

In the air, it keeps updating that map. It tracks its position by fusing GPS, IMU, odometry, camera, and LiDAR data with SLAM and a Kalman filter.

When something changes, like the weather or another aircraft getting in the way, it replans and sends the new route back through UTM. If it can't continue safely, it can loiter, land somewhere low-risk, return to base, or deploy a parachute.

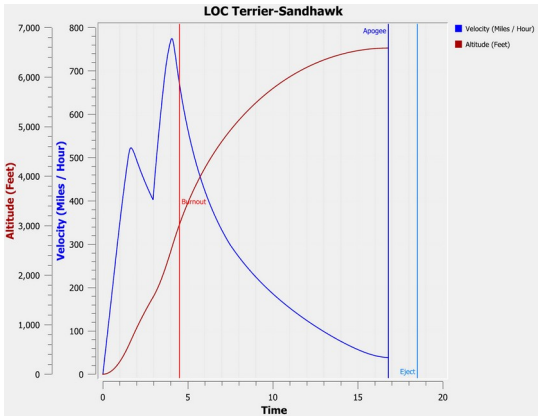


Modeled heart transfer, San Francisco General to Santa Clara Valley Medical Center

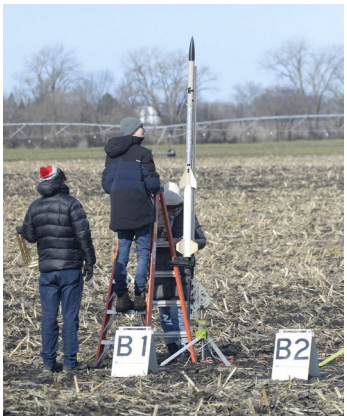
Full memo: ntrs.nasa.gov/citations/20250009029 (NASA/TM-20250009029)

TERRIER-SANDHAWK TWO-STAGE ROCKET

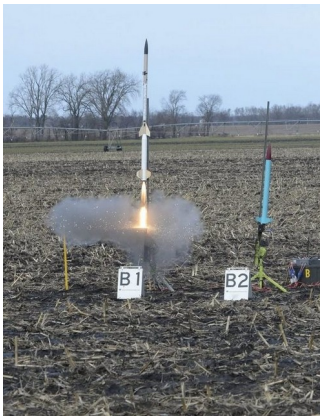
Builder and Flyer, Personal Project
January 2025



RockSim prediction, about 6,590 ft and 775 mph



Setting up on pad B1



Liftoff

What?

- My first two-stage high-power rocket, a LOC Precision 2.26" Sandhawk sustainer on a Terrier booster (76.9 in, 2,078 g)
- Flew on a CTI I297, staging to an H180, at Michiana Rocketry on January 31, 2025

How?

- Built the avionics for electronic staging, sustainer air-start, GPS tracking, and telemetry. The stages separate 1.0 s after booster burnout and the sustainer lights 0.7 s later
- Simulated the flight in **RockSim** to choose motors and check stability (2.40 static margin)
- Chose the staging delays to balance visibility from the ground, altitude, and stability in wind

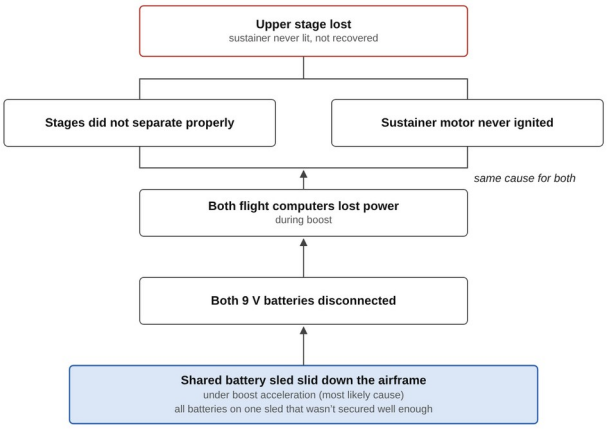
Results

- With the 0.7 s ignition delay included (RockSim leaves it out), predicted apogee rises from about 6,590 ft to 6,860 ft
- On the flight the stages didn't separate properly, the sustainer never lit, and the upper stage was lost
- Both 9 V batteries were on one sled that most likely slid under boost and disconnected them, which taught me the importance of redundant avionics

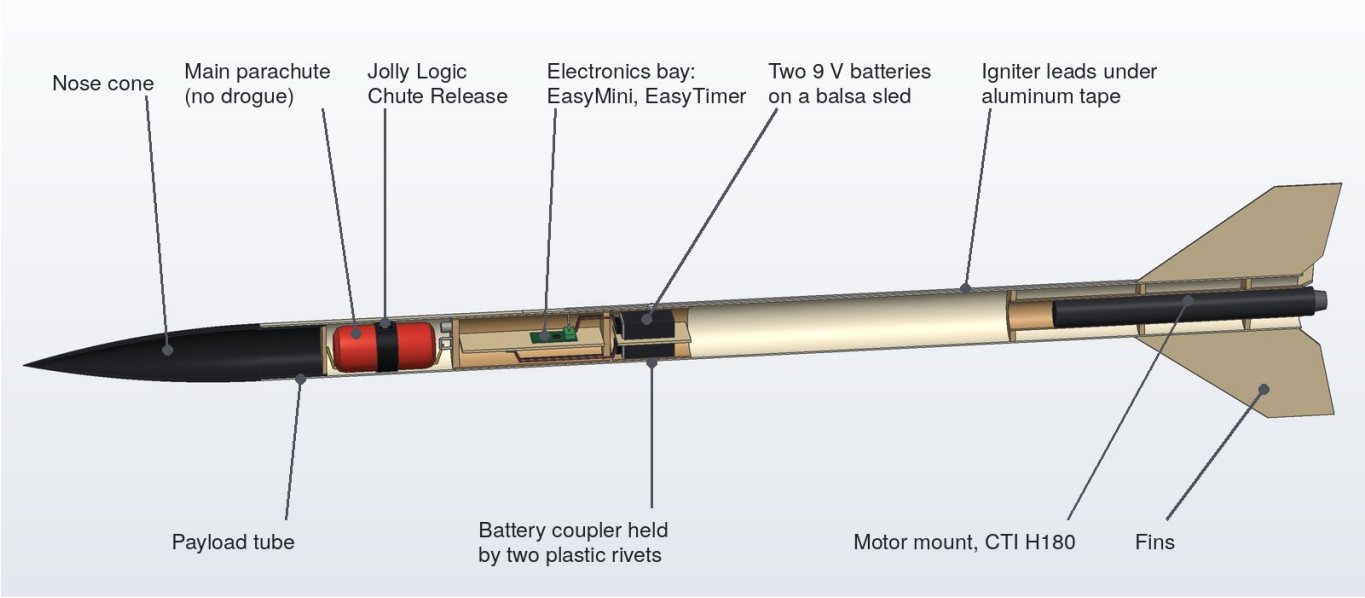
FAILURE ANALYSIS

The stages didn't separate properly and the sustainer never lit. Since both happened on the same flight, I looked for something that could cause both.

The sustainer carried only a main parachute, held closed by a Jolly Logic Chute Release in the payload tube. There was no drogue. Below the payload tube, the electronics bay held an Altus Metrum EasyMini for recovery and an EasyTimer for staging, and the EasyTimer's igniter leads ran down the outside of the airframe under aluminum tape to the sustainer motor. Both altimeters ran on 9 V batteries mounted to one balsa sled, inside a coupler held in by two plastic rivets.

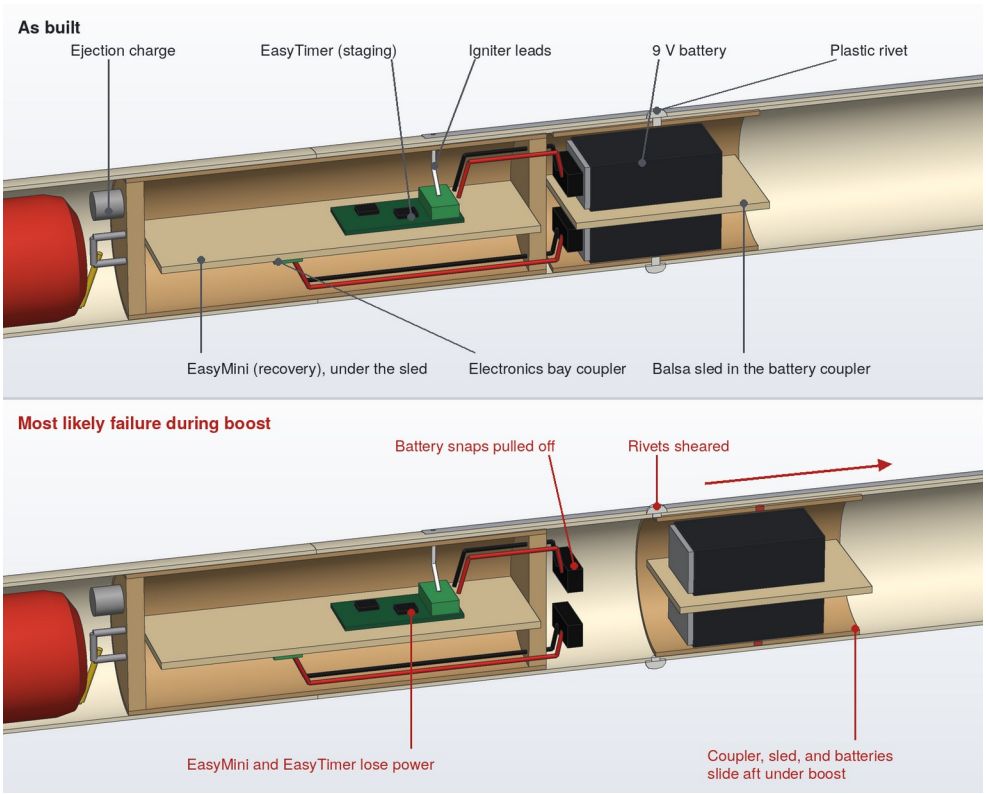


How one battery mount led to both failures



3-D model of the sustainer as flown, cut in half to show the inside. Positions are approximate.

My best explanation is that either the balsa sled broke or the rivets sheared. Under boost acceleration the batteries would then slide aft and pull loose from both altimeters, so all of the electronics lost power at once.

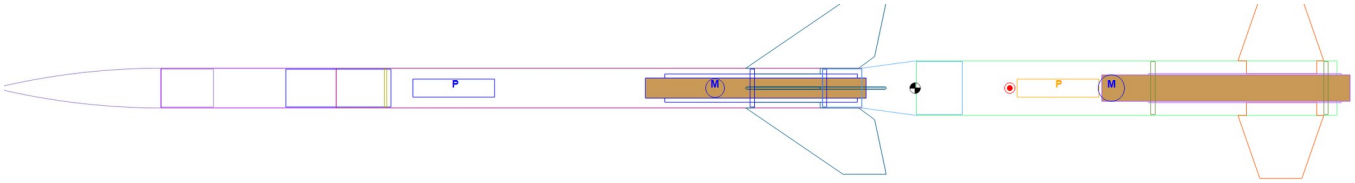


The electronics bay and battery coupler as built (top) and the most likely failure during boost (bottom)

This flight taught me the importance of redundant avionics systems. Having more than one flight computer didn't help, because they all depended on the same mount.

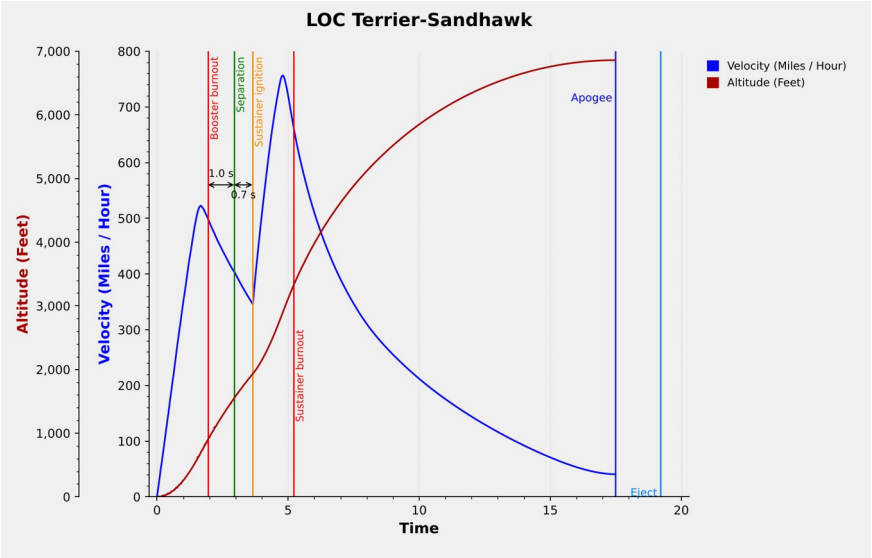
STAGING DELAY ANALYSIS

The EasyTimer let me set the staging delays myself, so I had to decide what they should be. I wanted a flight that looked good and went high. To me, looking good meant you could clearly see each event from the ground: booster burnout, then separation, then the sustainer lighting. Spacing them farther apart makes each one easier to see, and some delay can add altitude. But the longer the sustainer coasts without thrust, the slower it's going when it lights, and the easier it is for wind to push it off course. I still want to be able to fly in conditions that aren't perfect, so I settled on 1.0 s from booster burnout to separation and another 0.7 s to sustainer ignition.

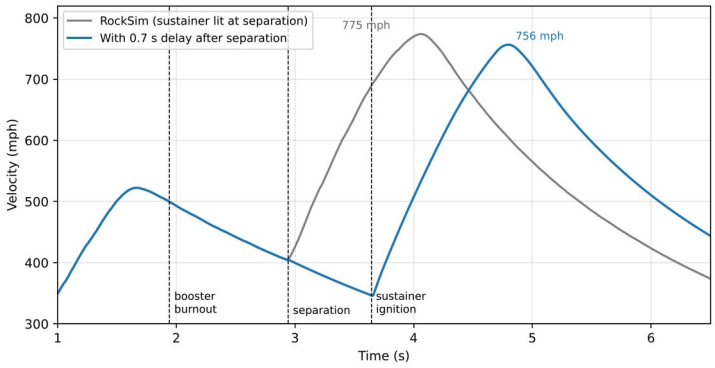


RockSim model of the full stack, sustainer on the left (CG 52.5 in, CP 57.9 in from the nose, static margin 2.40)

RockSim lights the sustainer at the moment the stages separate, so its prediction does not include the 0.7 s wait before sustainer ignition. To model the actual sequence, the upper stage was re-flown from separation with a two-dimensional point-mass model. The sustainer's drag-to-mass ratio was taken from RockSim's coast phase as a function of Mach number, and its ignition mass was fitted to the boost phase using the H180's certified thrust curve. Run with RockSim's own timing, the model matches RockSim's apogee within 3 ft and its peak speed within 0.3%.



Predicted flight with both delays. Booster burnout (1.9 s), separation (2.9 s), sustainer ignition (3.6 s), and sustainer burnout (5.2 s) are far enough apart to follow from the ground.



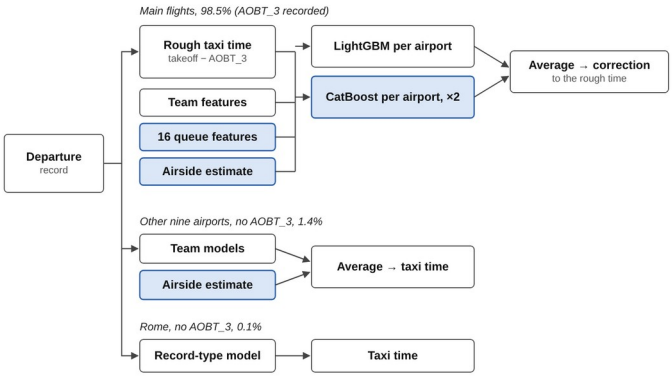
RockSim (gray) vs. delayed ignition (blue)

With the delay, the sustainer lights at 347 mph instead of 405 mph and peaks at 756 mph instead of 775 mph. Less of the burn happens near Mach 1, where drag is highest, so the predicted apogee rises from about 6,590 ft to 6,860 ft.

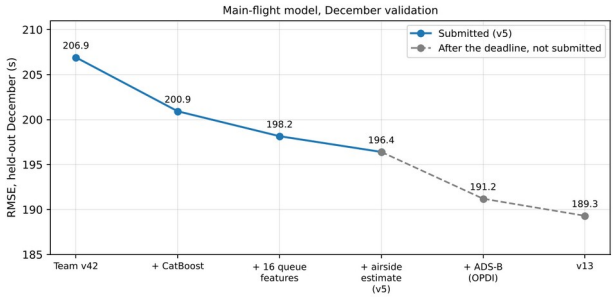
A 10% change in drag moves this by about 300 ft. Because the sustainer never lit on this flight, the prediction couldn't be checked against flight data.

EUROCONTROL PRC DATA CHALLENGE

Team Member, Embry-Riddle Team
September – October 2026



Team model, my additions in blue



Held-out error by version (gray points came after the deadline)

What?

- Predict aircraft taxi-out time at ten major European airports from about 2 million 2025 departures, scored by RMSE on January and July 2026 flights
- I joined Embry-Riddle's team "merry-llama" with very little Python or machine learning experience

How?

- I was responsible for the model for main flights, which covers 98.5% of departures
- Added **CatBoost** models alongside the team's LightGBM models
- Built 16 departure-queue features, like how many planes took off from the same runway in the previous 5 to 20 minutes
- Used **SHAP** on the 1,000 worst predictions, then trained an "airside" taxi-time estimate that doesn't rely on the reported off-block time
- Decided what counted as an improvement before each test, and kept notes on what worked and what didn't across 20+ experiments

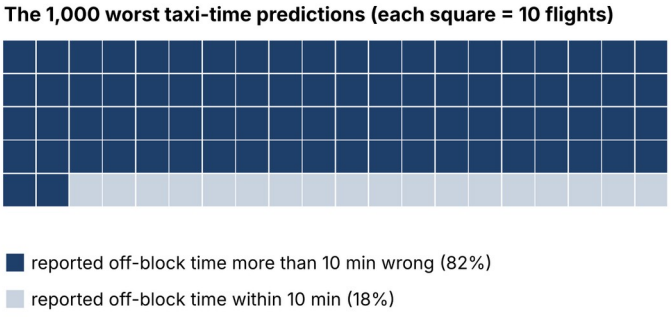
Results

- Lowered the team's official RMSE from 276.53 s to 267.34 s
- Taeyun Yoo built the team's best score (265.02 s) and the final pipeline on top of this version. The final pipeline still uses my CatBoost models and airside estimate
- Our team finished 47th of 94 on the final ranking, which used a different test set

APPROACH

I started from the team's LightGBM model for main flights and added one thing at a time, testing each change on held-out data. Before each test I decided what result would count as an improvement.

When I looked at the worst predictions with SHAP, most of them were flights whose reported off-block time was more than 10 minutes off. So I trained an airside estimate that doesn't use that time at all.



Worst 1,000 predictions

FINAL RESULTS

Our team finished 47th of 94, with an RMSE of 220.23 s. The final ranking was scored on February and June 2026 flights that never appeared on the leaderboard, so that number isn't directly comparable to the January and July scores above. The top three teams scored 176.16, 180.18, and 183.66 s.

After the challenge, I read the top three teams' write-ups. Most of the gap came from things they did that we didn't:

- Used public ADS-B tracks to see when each plane actually left its stand. That fixes the same off-block time problem my SHAP analysis found, and our final model didn't use ADS-B data at all.
- Modeled which recorded taxi times were really copied schedule times. One team also recovered unmatched flights from their arrival records at the destination.
- Averaged many neural networks or sequence models with their tree models. We only used gradient-boosted trees.

In my next data challenge or machine learning project, I'd:

- Look for the best raw data source before tuning the model
- Set aside a test set at the start and never use it to make decisions
- Find out early which small group of flights causes most of the error
- Try more than one kind of model from the start

Final ranking: ansperformance.eu/study/data-challenge/dc2026/ranking.html

Team code: github.com/taeyunyoo/prc-2026-merry-llama